BRAINWARE UNIVERSITY

School of Engineering

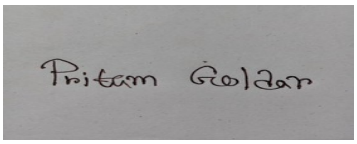
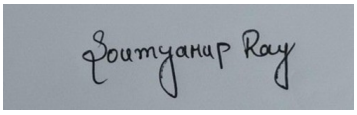
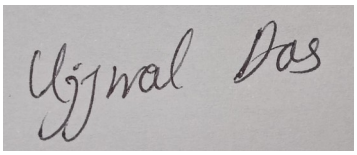
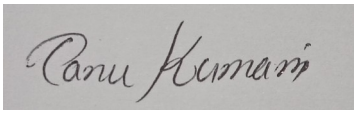
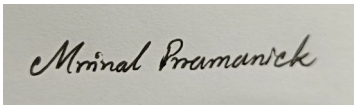
Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Mini Project (A.Y. 2026-27 ODD)

Course Code: BTD38101/BTY38101 **Course Name:** Mini Project-I

Program: B.Tech. CSE-DS/CYS [Batch: 2025-29]

S.N.	Roll No.	Student Code	Student Name	Signature
1.	25103610001	BWU/BCC/25/001	Pritam Golder	
2.	25103610002	BWU/BCC/25/002	Soumyarup Ray	
3.	25103610003	BWU/BCC/25/003	Ujjwal Das	
4.	25103610004	BWU/BCC/25/005	Tanu Kumari	
5.	25103610005	BWU/BCC/25/006	Mrinal Pramanick	

Project Title: Promix: A Fast, Secure and Cross-Platform Text Editor Built in Rust

Name & Signature of Project Supervisor

Dr. Suparna Panchanan

BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering – Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Understanding of Design / Research Methodology:

1. Introduction

A text editor is one of the most frequently used tools for students and developers, yet many popular editors are tied to one operating system, are resource-heavy, or need plugins and long configuration before basic features such as auto-closing brackets, fuzzy file search and good syntax colors work. Unsaved work can also be lost when an editor or the computer crashes.

Promix is a lightweight, cross-platform text editor written in Rust for Windows and Linux. Rust offers performance close to C and compile-time memory safety, which suits a tool that must be fast, stable and secure. The project also gives the team practical experience in data structures (rope-based text buffers), Unicode and encodings, file systems, concurrency, graphical interfaces, software testing and security.

2. Objectives

1. Build one editor from a single Rust codebase that runs on Windows and Linux.
2. Edit any file type safely: detect text encodings, preserve line endings, and handle binary files with a warning.
3. Provide automatic completion of ( ), [ ], { }, double quotes and single quotes, with auto-indentation and smart Tab.
4. Provide a built-in fuzzy file finder in the style of fzf (no external program needed), regular-expression search, project-wide search, find and replace, and line numbers.
5. Provide a beautiful built-in dark and light theme, syntax highlighting, and easy keyboard shortcuts with a command palette.
6. Make the editor stable and recover unsaved work automatically after a crash.
7. Distribute binaries and source code on GitHub, with an update notification and a separate updater program that uses Git.

3. Proposed Methodology

The team will follow an iterative, milestone-based approach over a 12-week schedule. Each milestone is implemented, checked by an automated quality gate (formatting, linter, tests, release build and dependency audit) and reviewed by the team before the next one begins.

- **Design:** a layered architecture: user interface, editor core (text buffer, cursor, undo and typing rules), services (file input and output, search, recovery, updates) and a small platform layer. Editing logic is kept separate from the interface so it can be tested without a window.
- **Technology:** Rust; egui/eframe for the interface; ropey for the text buffer; nucleo-matcher, ignore and regex for search; syntect for syntax highlighting; encoding_rs and chardetng for encodings; Git and GitHub for version control and release.
- **Work plan:** weeks 1 to 3: setup, text buffer and file handling; weeks 4 to 7: editor view, tabs, typing assistance, theme and highlighting; weeks 8 to 10: find and replace, fuzzy and project search, crash recovery and shortcuts; weeks 11 to 12: update system, security review, testing on Windows and Linux, report and demonstration.

- **Testing:** unit tests and property-based tests (random edits and keystrokes), fixture files (Windows and Linux line endings, non-UTF-8 text, binary data, very long lines), a headless self-test, a smoke test of the interface, and a manual checklist on real Windows and Linux machines.
- **Security and reliability:** safe Rust only; no crashes on bad input; size limits and cancellation for long tasks; background threads so the interface never freezes; Git started without a shell; atomic saves and private autosave backups; dependency checks with cargo-audit and cargo-deny.
- **Distribution:** ready-to-run binaries and source code on GitHub. The editor notifies the user of an update and a separate updater applies it with Git, because Windows cannot replace a program while it is running.

4. Expected Outcomes

- Working Promix binaries for Windows and Linux with all the features listed in the objectives.
- Complete source code on GitHub with documentation and a user guide that includes the keyboard shortcuts.
- An automated test suite, manual test results and a security review report.
- A 5,000-line file that opens and scrolls smoothly, with an interface that stays responsive during searches.
- No known crash or data-loss bug at submission, and no unresolved known vulnerability in the dependencies.
- A final project report and a live demonstration.
- **Future scope:** support for very large files, user settings and custom key bindings, plugins and language-server support, macOS support and automated CI/CD.

5. Conclusion

Promix aims to deliver a fast, reliable and good-looking editor that works the same way on Windows and Linux, while giving the team a complete learning experience in systems programming, interface design, testing and security. The milestone plan, quality checks and test strategy keep the project measurable and achievable within the semester.